

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

Code : `SELECT * FROM customers;`

Output

```
141 • SELECT *
142 FROM customers;
143
```

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

Code : `SELECT first_name, last_name, city FROM customers;`

Output

```
144 • SELECT
145     first_name,
146     last_name,
147     city
148 FROM customers;
149
```

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

Code : `SELECT DISTINCT category FROM products;`

Output

```
149
150 • SELECT DISTINCT category
151 FROM products;
152
```

Result Grid | Filter Rows:

	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema.
Explain why a Primary Key must be unique and NOT NULL.

Code : `SHOW KEYS FROM customers WHERE Key_name = 'PRIMARY';`

Output

```
153
154 • SHOW KEYS FROM customers
155 WHERE Key_name = 'PRIMARY';
156
157 • SHOW KEYS FROM products
158 WHERE Key_name = 'PRIMARY';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null
▶	customers	0	PRIMARY	1	customer_id	A	8	NULL	NULL	

Code : SHOW KEYS FROM products WHERE Key_name = 'PRIMARY';

Output

```
156
157 • SHOW KEYS FROM products
158 WHERE Key_name = 'PRIMARY';
159
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null
▶	products	0	PRIMARY	1	product_id	A	8	NULL	NULL	

Code : SHOW KEYS FROM orders WHERE Key_name = 'PRIMARY';

Output

```
159
160 • SHOW KEYS FROM orders
161 WHERE Key_name = 'PRIMARY';
162
```

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: 



	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	In
	orders	0	PRIMARY	1	order_id	A	10	NULL	NULL		BTI

Code : SHOW KEYS FROM order_items WHERE Key_name = 'PRIMARY';

Output

```
162
163 • SHOW KEYS FROM order_items
164 WHERE Key_name = 'PRIMARY';
165
```

Result Grid											Filter Rows:	Export:	Wrap Cell Content:
	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null			
▶	order_items	0	PRIMARY	1	item_id	A	15	NULL	NULL				

Explanation:

The primary key of each table is:

customers -> customer_id products -> product_id orders -> order_id order_items -> item_id

A primary key is used to identify each record in a table. Every value must be unique so that no two records are the same. It also cannot be NULL because every record must have its own ID.

Q5. What constraints are applied to the email column in the customers table?

What would happen if you tried to insert a duplicate email?

Code : INSERT INTO customers VALUES
(109,'Rahul','Kumar','aarav.s@email.com','Lucknow','Uttar Pradesh','2024-09-01',FALSE
);

Output



```
166 • INSERT INTO customers
167 VALUES
168 (
169 109,
170 'Rahul','Kumar','aarav.s@email.com',
171 'Lucknow','Uttar Pradesh','2024-09-01',FALSE
172 );
```

#	Time	Action	Message	Duration / Fetch
12	23:39:08	SHOW KEYS FROM order_items WHERE Key_name = 'PRIMARY'	1 row(s) returned	0.000 sec / 0.000 sec
13	23:39:49	INSERT INTO customers VALUES (109, 'Rahul','Kumar','aarav.s@email....	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'	

Explanation:

The email column has two constraints:

1. UNIQUE
2. NOT NULL

UNIQUE makes sure the same email cannot be stored more than once.

NOT NULL makes sure every customer has an email address.

If I try to insert an email that already exists, MySQL shows a duplicate entry error and the new record is not added.

Q6. Try inserting a product with unit_price = -50.
What happens and which constraint prevents it?
Write both the INSERT statement and explain the error.

Code : INSERT INTO products VALUES (209,'Test
Product','Electronics','TestBrand',-50,100);

Output

```
174 • INSERT INTO products
175 VALUES
176 (
177 209,'Test Product','Electronics','TestBrand',-50,100
178 );
```

Context Help Sn

Output

❏ Action Output ▼

#	Time	Action	Message
✖ 13	23:39:49	INSERT INTO customers VALUES (109, 'Rahul','Kumar','aarav.s@email....	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers...
✖ 14	23:41:19	INSERT INTO products VALUES (209,'Test Product','Electronics','TestB...	Error Code: 3819. Check constraint 'products_chk_1' is violated.

Explanation:

This query will fail because the product price cannot be negative.

The unit_price column has a CHECK constraint that only allows values greater than zero. Since I entered -50, MySQL rejects the record and displays an error.

Section B - Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

Code : `SELECT * FROM orders WHERE status = 'Delivered';`

Output

181 • `SELECT * FROM orders WHERE status = 'Delivered';`

Result Grid |   Filter Rows: | Edit:    | Export/Import: 

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
•	NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

Code : `SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;`

Output

183 • `SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;`

Result Grid

	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
*	NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Code : `SELECT *FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'AND state = 'Maharashtra';`

Output

[illegible]

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Code : `SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';`

Output

```
190 • SELECT *
191 FROM orders
192 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
193 AND status <> 'Cancelled';
194
```

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
* NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index `idx_orders_date` does.
How would it improve the performance of a query that filters orders by `order_date`?
Write a sample query that would benefit from this index.

Code : `SELECT * FROM orders WHERE order_date = '2024-08-15';`

Output

```
195 • SELECT *
196 FROM orders
197 WHERE order_date = '2024-08-15';
```

order_id	customer_id	order_date	status	total_amount
1006	105	2024-08-15	Delivered	5898.00
* NULL	NULL	NULL	NULL	NULL

Explanation:

The `idx_orders_date` index is created on the `order_date` column.

When a query searches records using `order_date`, MySQL can find the matching rows much faster instead of checking every row in the table.

This helps improve performance, especially when the table contains a large number of records.

Q12. If you run:

```
SELECT * FROM customers WHERE YEAR(join_date) = 2024;
```

would the index on join_date be used?

Explain why or why not, and rewrite the query to be index-friendly (SARGable).

-- Query using YEAR()

Code :

```
SELECT * FROM customers WHERE YEAR(join_date) = 2024;
```

Output

```
200 • SELECT *
201 FROM customers
202 WHERE YEAR(join_date) = 2024;
203
```

Result Grid							
Filter Rows: [] Edit: [] Export/Import: [] Wrap Cell Content: []							
customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
[]							

-- Index-friendly query

Code :

```
SELECT * FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31';
```

```
205 • SELECT *
206 FROM customers
207 WHERE join_date BETWEEN '2024-01-01'
208 AND '2024-12-31';
```

Result Grid							
Filter Rows: [] Edit: [] Export/Import: [] Wrap Cell Content: []							
customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
[]							

Output

Explanation:

Using YEAR(join_date) makes MySQL calculate the year for every row before filtering the data.

Because of this, the database may not use the index efficiently.

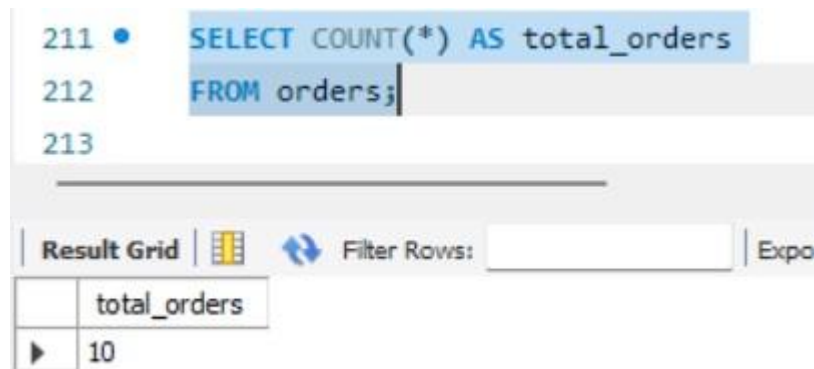
Using a date range allows MySQL to search directly through the indexed values, so the query performs better.

Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

Code : `SELECT COUNT(*) AS total_orders FROM orders;`

Output



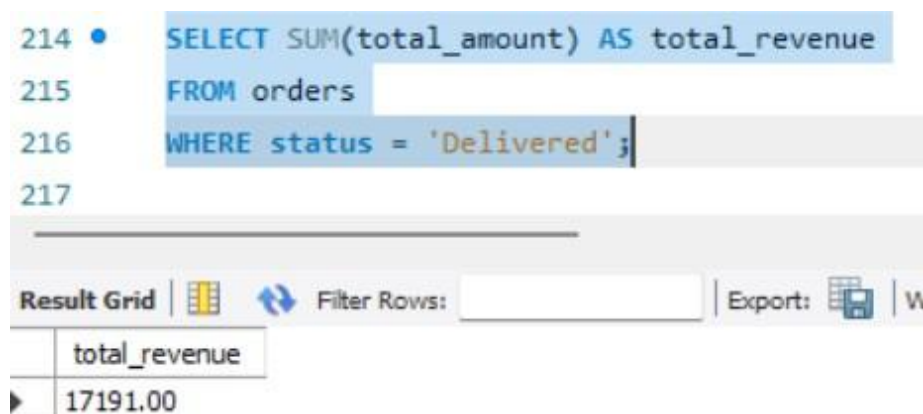
```
211 • SELECT COUNT(*) AS total_orders
212 FROM orders;
213
```

	total_orders
▶	10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

Code : `SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';`

Output



```
214 • SELECT SUM(total_amount) AS total_revenue
215 FROM orders
216 WHERE status = 'Delivered';
217
```

	total_revenue
▶	17191.00

Q15. Calculate the average unit_price of products in each category.

Code : `SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category;`

Output

```
218 • SELECT
219     category,
220     AVG(unit_price) AS average_price
221 FROM products
222 GROUP BY category;
```

category	average_price
Clothing	2699.000000
Electronics	2224.000000
Home	949.000000

Q16. For each order status, find the count of orders and the total revenue.

Sort the result by total revenue in descending order.

Code : `SELECT status,COUNT(*) AS total_orders,SUM(total_amount) AS total_revenue FROM orders GROUP BY status ORDER BY total_revenue DESC;`

Output

```
224 • SELECT
225     status,
226     COUNT(*) AS total_orders,
227     SUM(total_amount) AS total_revenue
228 FROM orders
229 GROUP BY status
230 ORDER BY total_revenue DESC;
```

status	total_orders	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

Code : `SELECT category, MAX(unit_price) AS highest_price, MIN(unit_price) AS lowest_price FROM products GROUP BY category;`

Output

```
231
232 • SELECT
233     category,
234     MAX(unit_price) AS highest_price,
235     MIN(unit_price) AS lowest_price
236 FROM products
237 GROUP BY category;
```

238

category	highest_price	lowest_price
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000.

(Hint: Use HAVING clause)

Code : `SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category HAVING AVG(unit_price) > 2000;`

Output

```
238
239 • SELECT
240     category,
241     AVG(unit_price) AS average_price
242 FROM products
243 GROUP BY category
244 HAVING AVG(unit_price) > 2000;
```

245

category	average_price
Clothing	2699.000000
Electronics	2224.000000

Section D - Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's

first_name and last_name.

Show: order_id, order_date, first_name, last_name, total_amount.

```
Code : SELECT o.order_id, o.order_date, c.first_name, c.last_name,  
o.total_amount FROM orders o INNER JOIN customers c ON o.customer_id =  
c.customer_id;
```

Output

```
247 • SELECT
248     o.order_id,
249     o.order_date,
250     c.first_name,
251     c.last_name,
252     o.total_amount
253 FROM orders o
254 INNER JOIN customers c
255 ON o.customer_id = c.customer_id;
```

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

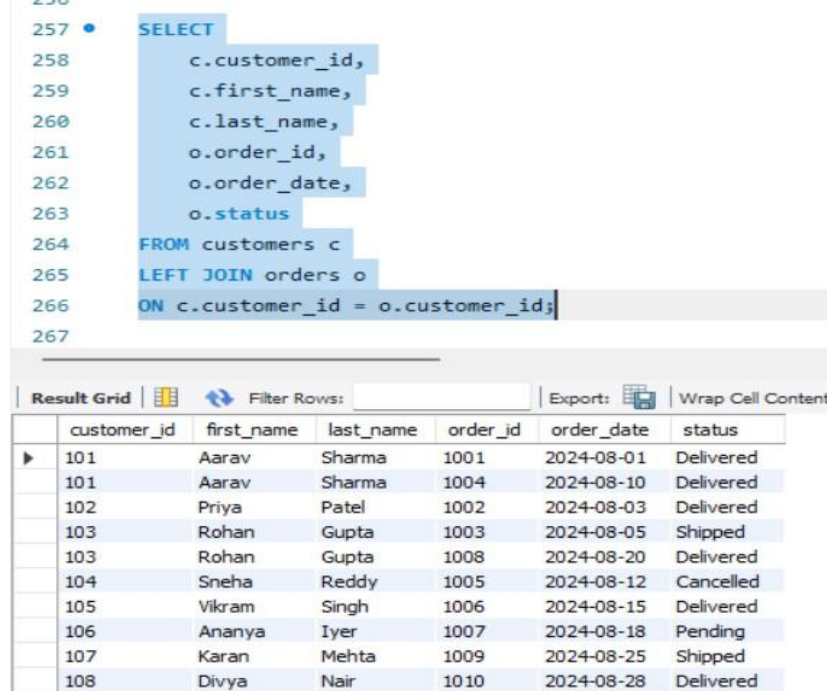
Result 28 x

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any).

Customers with no orders should still appear with NULL values for order columns.

Code : `SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.status FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;`

Output



The screenshot shows a SQL query editor with the following query:

```
257 • SELECT
258     c.customer_id,
259     c.first_name,
260     c.last_name,
261     o.order_id,
262     o.order_date,
263     o.status
264 FROM customers c
265 LEFT JOIN orders o
266 ON c.customer_id = o.customer_id;
267
```

Below the query editor is a 'Result Grid' showing the output of the query. The grid has columns: customer_id, first_name, last_name, order_id, order_date, and status. The data is as follows:

	customer_id	first_name	last_name	order_id	order_date	status
▶	101	Aarav	Sharma	1001	2024-08-01	Delivered
	101	Aarav	Sharma	1004	2024-08-10	Delivered
	102	Priya	Patel	1002	2024-08-03	Delivered
	103	Rohan	Gupta	1003	2024-08-05	Shipped
	103	Rohan	Gupta	1008	2024-08-20	Delivered
	104	Sneha	Reddy	1005	2024-08-12	Cancelled
	105	Vikram	Singh	1006	2024-08-15	Delivered
	106	Ananya	Iyer	1007	2024-08-18	Pending
	107	Karan	Mehta	1009	2024-08-25	Shipped
	108	Divya	Nair	1010	2024-08-28	Delivered

Q21. Write a query using JOINS across three tables (orders →

order_items → products)

to show:

order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Code : `SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct FROM orders o INNER JOIN order_items oi ON o.order_id = oi.order_id INNER JOIN products p ON oi.product_id = p.product_id;`

Output

```
268 • SELECT
269     o.order_id,
270     p.product_name,
271     oi.quantity,
272     oi.unit_price,
273     oi.discount_pct
274 FROM orders o
275 INNER JOIN order_items oi
276 ON o.order_id = oi.order_id
277 INNER JOIN products p
278 ON oi.product_id = p.product_id;
279
```

Result Grid | Filter Rows: | Export: | Wrap Cell

	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00

Result 30 x

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN

with an example from this schema.

When would you use a FULL OUTER JOIN?

Explanation:

LEFT JOIN returns all records from the left table and the matching records from the right table.

Example: If we use customers as the left table, every customer will be displayed even if they have not placed any order.

RIGHT JOIN works in the opposite way. It returns all records from the right table and the matching records from the left table.

A FULL OUTER JOIN returns all records from both tables. It is useful when we want to see both matching and non-matching records from both sides.

Note: MySQL does not support FULL OUTER JOIN directly. It can be achieved by combining LEFT JOIN and RIGHT JOIN using UNION.

Q23. Identify all Foreign Key relationships in the schema.

Explain what would happen if you tried to insert an order with `customer_id = 999` (which doesn't exist in customers).

Foreign Key Relationships

`orders.customer_id` references `customers.customer_id`

`order_items.order_id` references `orders.order_id`

`order_items.product_id` references `products.product_id`

If `customer_id = 999` does not exist in the `customers` table, MySQL will not allow the record to be inserted.

It will return a Foreign Key Constraint error because every order must belong to an existing customer.

-- Example

```
INSERT INTO orders VALUES ( 1011, 999, '2024-09-01', 'Pending', 1598.00 );
```

Section E - Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Code : `SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN 'Budget' WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range' ELSE 'Premium' END AS price_tier FROM products;`

Output

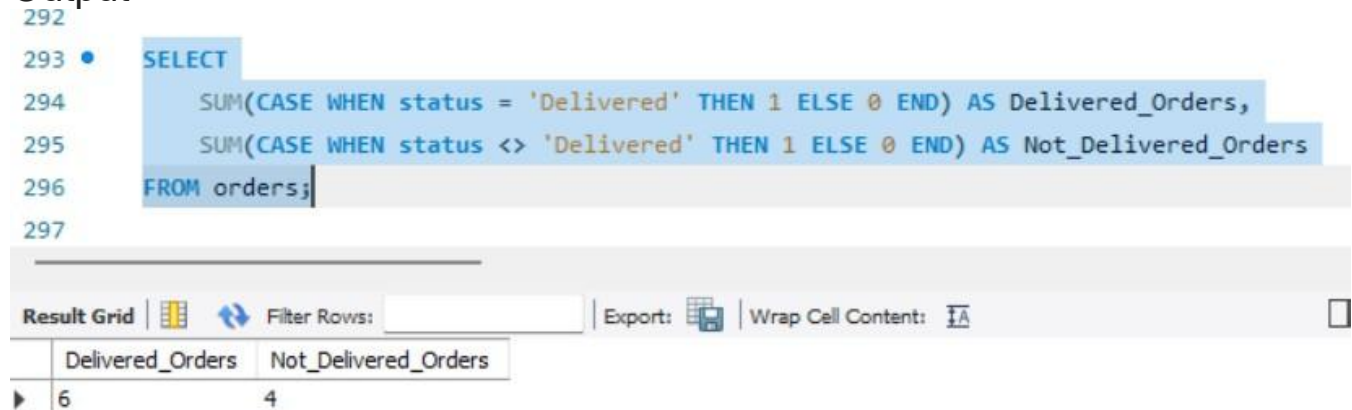
```
282
283 • SELECT
284     product_name,
285     unit_price,
286     CASE
287         WHEN unit_price < 1000 THEN 'Budget'
288         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
289         ELSE 'Premium'
290     END AS price_tier
291 FROM products;
292
```

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Code : `SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Orders, SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS Not_Delivered_Orders FROM orders;`

Output



```
292
293 • SELECT
294     SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Orders,
295     SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS Not_Delivered_Orders
296 FROM orders;
297
```

Delivered_Orders	Not_Delivered_Orders
6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Explanation:

A - Atomicity

Atomicity means all steps of a transaction are completed together. If one step fails, the whole transaction is cancelled.

Example: During a bank transfer, if the money is deducted from one account but cannot be added to the other account, the transaction is rolled back.

C - Consistency

Consistency means the database always stays in a valid state. The data should follow all rules and constraints.

Example: A customer order cannot be created with a customer_id that does not exist in the customers table.

I - Isolation

Isolation means multiple transactions can run at the same time without affecting each other.

Example: If two customers place orders at the same time, both transactions should complete correctly without mixing data.

D - Durability

Durability means once a transaction is committed, the changes are permanently saved.

Example: If the system crashes after an order is placed, the order should still be available after restarting the database.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order
2. Insert two order items
3. Update the stock quantity
4. If any step fails, rollback the transaction.

Otherwise, commit.

Code : START TRANSACTION;

```
INSERT INTO orders VALUES ( 1011, 102, CURDATE(), 'Pending', 1598.00 );
```

```
INSERT INTO order_items VALUES ( 5016, 1011, 206, 1, 1299.00, 0 );
```

```
INSERT INTO order_items VALUES ( 5017, 1011, 208, 1, 299.00, 0 );
```

```
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206;
```

```
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208;
```

```
COMMIT;
```

If any statement fails before COMMIT, the transaction should be rolled back.

Example:

```
ROLLBACK;
```